Project Report

Hamming Code (7,4) Generation for error detection in assembly using MIPS32 pipeline architecture

By Sharthak Raj
B.E. Electronics and Communication Engineering
KLE Technological University
Semester: VI, 2025-26

1.Introduction

Error detection and correction are essential requirements in modern digital communication systems to ensure reliable data transmission. During transmission, data bits may get corrupted due to noise, interference, or hardware faults, resulting in communication errors. To overcome this problem, error detection and correction techniques are used. Among these techniques, Hamming Code is one of the most efficient methods for detecting and correcting single-bit errors.

This project presents the design and implementation of **Hamming Code (7,4) generation and error detection using a pipelined MIPS32 processor implemented in Verilog HDL**. The system accepts a 4-bit binary input and generates a 7-bit Hamming encoded output by calculating the required parity bits. The generated Hamming code is then analysed to determine whether an error exists in the received data. Syndrome bits are calculated using XOR operations, and the final syndrome value identifies the exact position of the error bit.

The processor architecture used in this project is based on a **5-stage MIPS32 pipeline architecture**, consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages. Two separate clocks are used for parallel execution of instructions, improving the efficiency of the system.

The project is implemented using Verilog HDL and verified through software simulation using waveform analysis and terminal outputs. Hardware implementation is also demonstrated using LEDs and LCD displays for real-time visualization of results. The project successfully demonstrates Hamming code generation, syndrome calculation, and error detection using pipelined processor architecture.

2. OBJECTIVES OF THE PROJECT

The main objective of this project is to design and implement a reliable error detection system using Hamming Code (7,4) on a pipelined MIPS32 processor architecture. The project focuses on understanding digital communication error control mechanisms and implementing them using assembly language programming and Verilog HDL.

The specific objectives of the project are:

• To generate Hamming Code (7,4) from 4-bit input data

The system accepts a 4-bit binary input and generates a corresponding 7-bit Hamming encoded output by calculating parity bits using XOR operations.

• To implement single-bit error detection

The project detects whether an error has occurred in the received Hamming code by calculating syndrome bits.

• To identify the exact error position

The syndrome value generated from parity checking helps determine the exact location of the erroneous bit in the received code.

• To implement the design using MIPS32 pipeline architecture

The complete Hamming code generation and error detection process is implemented using a 5-stage pipelined MIPS32 processor.

• To understand assembly language programming

The project uses ALP instructions such as AND, XOR, OR, SLL, and SRL for bit manipulation and parity generation.

• To perform hardware implementation using Verilog HDL

The processor, LCD controller, and top-level modules are designed and implemented in Verilog HDL.

• To verify results through software and hardware simulation

Waveforms, terminal outputs, LEDs, and LCD displays are used to validate the correctness of the generated Hamming code and syndrome outputs.

• To study the advantages of pipelined processor architecture

The project demonstrates how pipelining improves instruction execution efficiency through parallel processing of instructions.

3. HAMMING CODE (7,4) THEORY

Hamming Code is an error detection and correction technique used in digital communication systems to improve data reliability. It is capable of detecting and correcting single-bit errors that may occur during data transmission.

In Hamming Code (7,4), four data bits are converted into a seven-bit encoded word by adding three parity bits. The parity bits are placed at positions that are powers of two, while the remaining positions are occupied by data bits.

The seven-bit Hamming code consists of:

- 4 Data Bits
- 3 Parity Bits

Thus,

$$\text{Total bits} = 4 + 3 = 7$$

The parity bits are placed at positions:

- Position 1 → P1
- Position 2 → P2
- Position 4 → P4

The data bits are placed at:

- Position 3 → D1
- Position 5 → D2
- Position 6 → D3
- Position 7 → D4

The final arrangement becomes:

Position	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Bit Stored	P1	P2	D1	P4	D2	D3	D4

The parity bits are generated using XOR operations.

Parity Bit Equations

Parity bit P1 checks positions:
1, 3, 5, 7

Therefore,

$$P1 = D1 \oplus D2 \oplus D4$$

Parity bit P2 checks positions:
2, 3, 6, 7

Therefore,

$$P2 = D1 \oplus D3 \oplus D4$$

Parity bit P4 checks positions:
4, 5, 6, 7

Therefore,

$$P4 = D2 \oplus D3 \oplus D4$$

The XOR operation is used because:

- XOR outputs 0 when inputs are same
- XOR outputs 1 when inputs are different

This property makes XOR suitable for parity generation and error detection.

4. HAMMING CODE STRUCTURE

The Hamming Code (7,4) structure is designed such that parity bits are inserted into specific positions within the transmitted codeword. These parity bits help in checking groups of bits and identifying any transmission errors.

The arrangement of bits in the Hamming Code is shown below:

Bit Position	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Bit Type	P1	P2	D1	P4	D2	D3	D4

Where:

- P1, P2, and P4 are parity bits
- D1, D2, D3, and D4 are data bits

Working of Parity Bits

P1 Checks:

Positions 1, 3, 5, and 7

That means P1 checks:

- D1
- D2
- D4

P2 Checks:

Positions 2, 3, 6, and 7

That means P2 checks:

- D1
- D3
- D4

P4 Checks:

Positions 4, 5, 6, and 7

That means P4 checks:

- D2
- D3
- D4

This arrangement allows the receiver to identify the exact bit position where an error occurs.

For example:
If syndrome = $101_2 = 5_{10}$, then the error is located at bit position 5.

Thus, the Hamming code not only detects the presence of an error but also identifies the exact error location.

5. HAMMING CODE GENERATION PROCESS

The Hamming code generation process begins with a 4-bit binary input. The processor extracts each bit individually and calculates the parity bits using XOR operations. After parity generation, all bits are arranged in the proper Hamming code structure to create the final 7-bit encoded output.

The complete process consists of the following steps:

Step 1: Input Data Loading

The 4-bit binary input is loaded into a register using ADDI instruction.

Example:
If input data = 1001

Then:

- D1 = 1
- D2 = 0
- D3 = 0
- D4 = 1

Step 2: Data Bit Extraction

The processor extracts each bit individually using:

- Shift Right Logical (SRL)
- Bitwise AND operations

Mask value 1 is used to isolate individual bits.

Step 3: Parity Bit Calculation

Using XOR operations:

$$P1 = D1 \oplus D2 \oplus D4$$
$$P2 = D1 \oplus D3 \oplus D4$$
$$P4 = D2 \oplus D3 \oplus D4$$

Step 4: Hamming Code Formation

The bits are arranged in the order:

Position	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Stored Bit	P1	P2	D1	P4	D2	D3	D4

For input:

1001

Generated parity bits:

- P1 = 0
- P2 = 0
- P4 = 1

Final Hamming Code:

0011001

which is decimal 25.

Step 5: Output Storage

The generated 7-bit Hamming code is stored in the output register and displayed through LEDs and LCD display.

6. ERROR DETECTION USING SYNDROME CALCULATION

After generating and transmitting the 7-bit Hamming code, errors may occur during transmission due to noise or signal disturbances. To identify whether the received data contains an error, syndrome calculation is performed.

The syndrome bits are generated by recalculating parity over specific bit groups in the received Hamming code. These syndrome bits help determine whether an error exists and identify the exact position of the erroneous bit.

The syndrome consists of three bits:

$$S = (S3\ S2\ S1)$$

Where:

- S1 checks parity group 1
- S2 checks parity group 2
- S3 checks parity group 4

Syndrome Bit Calculation

Calculation of S1

S1 checks bit positions:
1, 3, 5, and 7

Therefore,

$$S1 = b1 \oplus b3 \oplus b5 \oplus b7$$

Calculation of S2

S2 checks bit positions:
2, 3, 6, and 7

Therefore,

$$S2 = b2 \oplus b3 \oplus b6 \oplus b7$$

Calculation of S3

S3 checks bit positions:
4, 5, 6, and 7

Therefore,

$$S3 = b4 \oplus b5 \oplus b6 \oplus b7$$

Syndrome Interpretation

The three syndrome bits are combined to form a binary number that represents the error position.

Syndrome	**Meaning**
000	No Error
001	Error in Bit 1
010	Error in Bit 2
011	Error in Bit 3
100	Error in Bit 4
101	Error in Bit 5
110	Error in Bit 6
111	Error in Bit 7

If the syndrome value is 000, the received Hamming code is correct. Any non-zero syndrome indicates the exact position of the error bit.

Example of Error Detection

Consider transmitted Hamming code:

$$0110011$$

Suppose during transmission, bit 5 changes from 0 to 1.

Received code becomes:

$$0111011$$

Now syndrome bits are calculated:

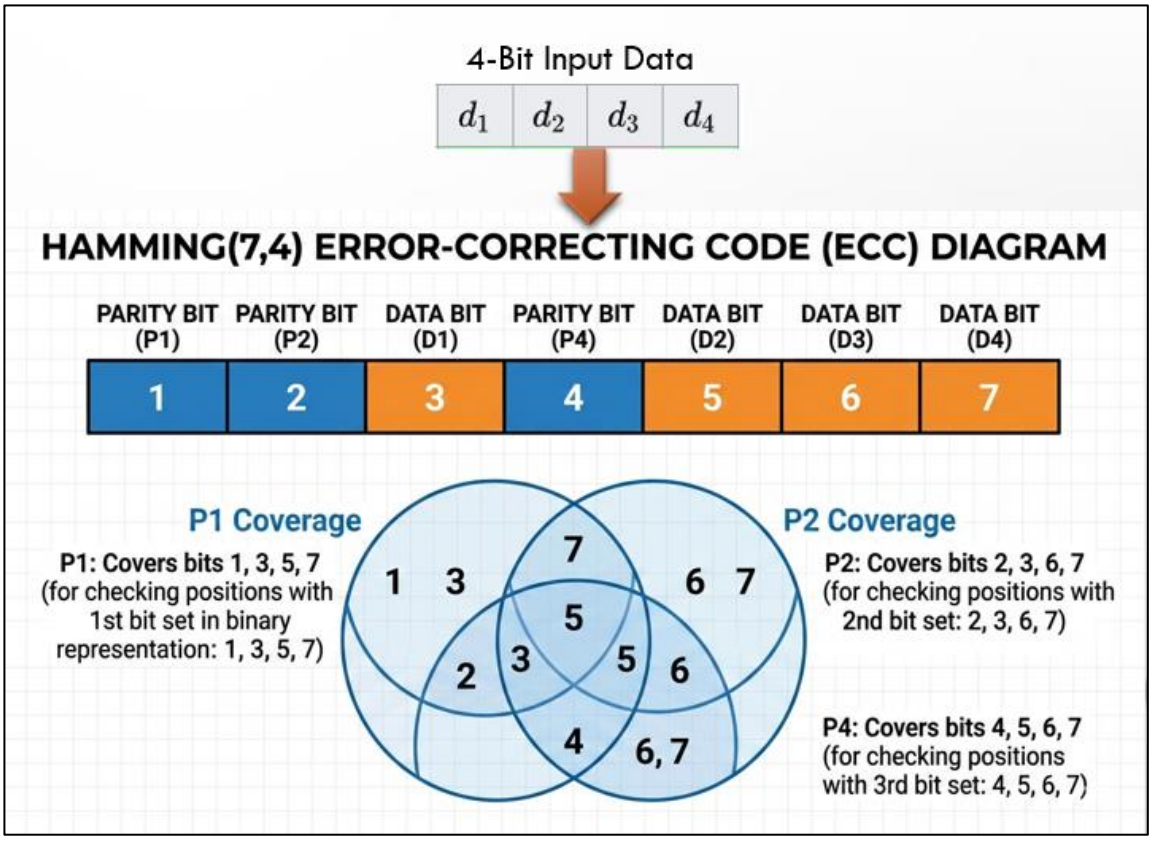
$$S3 = 1$$
$$S2 = 0$$
$$S1 = 1$$

Therefore,

$$S = 101_2$$
$$101_2 = 5_{10}$$

Hence, the error is present at bit position 5.

This method allows accurate single-bit error detection in digital communication systems.



7. MIPS32 PIPELINE ARCHITECTURE

The processor used in this project is based on the MIPS32 pipeline architecture. Pipelining is a technique used in processors to improve instruction execution speed by allowing multiple instructions to execute simultaneously in different stages.

Instead of completing one instruction fully before starting the next instruction, pipelining divides instruction execution into stages. Each stage performs a specific task, and all stages operate in parallel.

The processor implemented in this project uses a **5-stage pipeline architecture**.

Five Pipeline Stages

1. Instruction Fetch (IF)

In this stage:

- The instruction is fetched from instruction memory.
- The Program Counter (PC) contains the address of the instruction.
- PC is incremented for the next instruction.

Operations performed:

- Fetch instruction from memory
- Update PC value

2. Instruction Decode (ID)

In this stage:

- The fetched instruction is decoded.
- Source registers are identified.
- Required operands are read from register memory.

Operations performed:

- Instruction decoding
- Register reading
- Immediate value generation

3. Execute (EX)

In this stage:

- Arithmetic and logical operations are performed using the ALU.
- XOR, AND, OR, SRL, and SLL operations required for Hamming code generation are executed here.

Operations performed:

- ALU calculations
- Shift operations
- XOR parity calculations

4. Memory Access (MEM)

In this stage:

- Data memory operations are performed if required.
- For this project, memory access mainly supports pipeline transfer operations.

Operations performed:

- Memory read/write
- Data transfer

5. Write Back (WB)

In this stage:

- The final result is written back into the destination register.

Operations performed:

- Register update
- Output storage

Dual Clock Mechanism

The processor uses two clocks:

Clock	Pipeline Stages
clk1	IF, EX, WB
clk2	ID, MEM

This dual-clock arrangement prevents timing conflicts and improves synchronization between stages.

Advantages of Pipeline Architecture

• Increased Instruction Throughput

Multiple instructions execute simultaneously.

• Faster Execution

Parallel execution reduces total execution time.

• Efficient Hardware Utilization

All processor stages remain active.

• Improved System Performance

Better speed compared to non-pipelined processors.

Working of Pipeline in the Project

In this project:

- One instruction fetches input data.
- Another instruction extracts bits.
- Another instruction calculates parity simultaneously.

Thus, several ALP instructions execute in parallel through different pipeline stages, improving overall efficiency of Hamming code generation and error detection.

8. PIPELINE WORKING

Pipeline execution allows overlapping of instructions so that multiple instructions are processed simultaneously. Each instruction moves through the five pipeline stages sequentially.

For example:

Clock Cycle	Instruction 1	Instruction 2	Instruction 3
1	IF		
2	ID	IF	
3	EX	ID	IF
4	MEM	EX	ID
5	WB	MEM	EX
6		WB	MEM
7			WB

This overlapping significantly improves processor performance.

In this project:

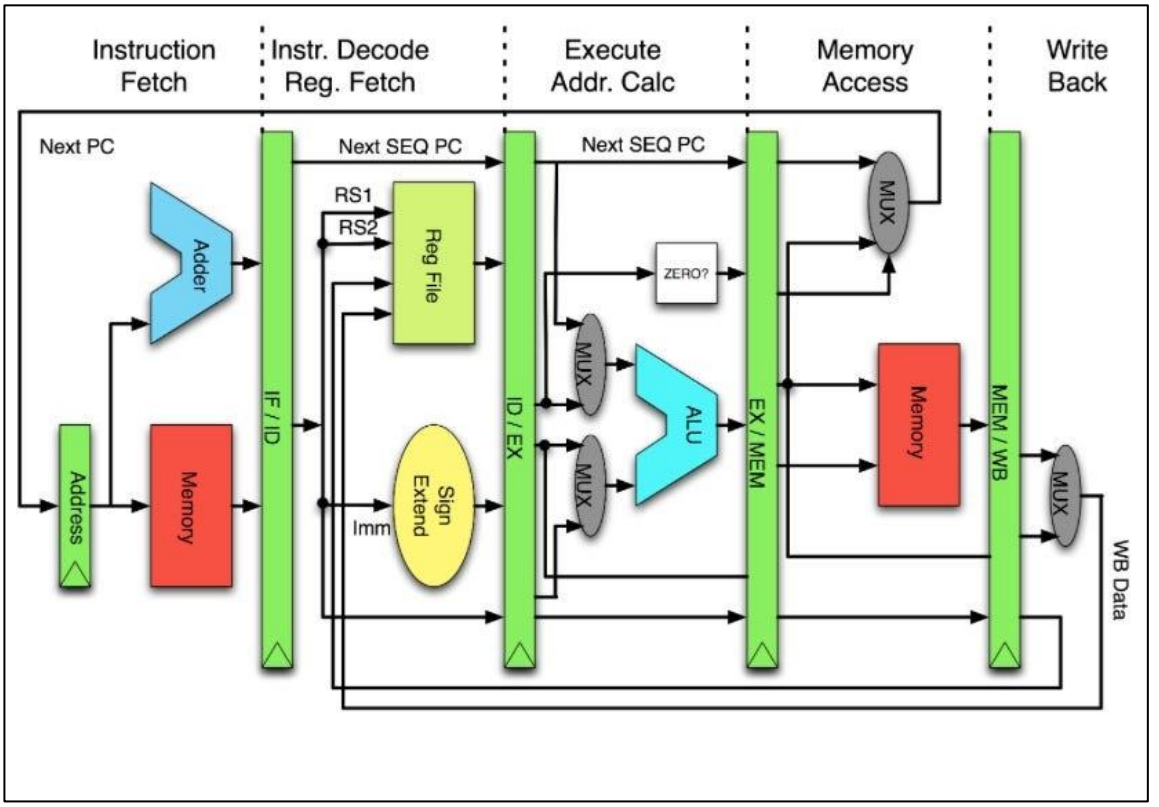
- Instruction Fetch reads ALP instructions from memory.
- Decode stage identifies registers and operations.
- Execute stage performs XOR and shift operations.
- Write Back stage stores parity and syndrome results.

Pipeline registers are used between stages to temporarily store intermediate values.

The important pipeline registers used are:

- IF/ID Register
- ID/EX Register
- EX/MEM Register
- MEM/WB Register

These registers maintain synchronization between stages and ensure smooth data transfer throughout the processor.



9. ASSEMBLY LANGUAGE PROGRAM FOR HAMMING CODE CONVERSION

The Assembly Language Program (ALP) used in this project performs the generation of Hamming Code (7,4) from a 4-bit binary input. The program is executed on the pipelined MIPS32 processor and uses logical and shift operations for parity generation and bit arrangement.

```
Mem[0]  = 32'h280a0001; Mem[1]  = 32'h280b0002;
Mem[2]  = 32'h280c0004; Mem[3]  = 32'h280d0008;
Mem[4]  = 32'h0ce77800; Mem[5]  = 32'h0ce77800;
Mem[6]  = 32'h0ce77800; Mem[7]  = 32'h0ce77800;
Mem[8]  = 32'h090a0800; Mem[9]  = 32'h0ce77800;
Mem[10] = 32'h0ce77800; Mem[11] = 32'h0ce77800;
Mem[12] = 32'h090b1000; Mem[13] = 32'h0ce77800;
Mem[14] = 32'h0ce77800; Mem[15] = 32'h0ce77800;
Mem[16] = 32'h544a1000; Mem[17] = 32'h090c1800;
Mem[18] = 32'h0ce77800; Mem[19] = 32'h0ce77800;
Mem[20] = 32'h0ce77800; Mem[21] = 32'h546b1800;
Mem[22] = 32'h0ce77800; Mem[23] = 32'h0ce77800;
Mem[24] = 32'h0ce77800; Mem[25] = 32'h090D2000;
Mem[26] = 32'h280d0003; Mem[27] = 32'h0ce77800;
Mem[28] = 32'h0ce77800; Mem[29] = 32'h0ce77800;
Mem[30] = 32'h548D2000; Mem[31] = 32'h0ce77800;
Mem[32] = 32'h0ce77800; Mem[33] = 32'h0ce77800;
Mem[34] = 32'h1883E800; Mem[35] = 32'h0ce77800;
Mem[36] = 32'h0ce77800; Mem[37] = 32'h0ce77800;
Mem[38] = 32'h1BA1E800; Mem[39] = 32'h0ce77800;
Mem[40] = 32'h0ce77800; Mem[41] = 32'h0ce77800;
Mem[42] = 32'h1882F000; Mem[43] = 32'h0ce77800;
Mem[44] = 32'h0ce77800; Mem[45] = 32'h0ce77800;
Mem[46] = 32'h1BC1F000; Mem[47] = 32'h0ce77800;
Mem[48] = 32'h0ce77800; Mem[49] = 32'h0ce77800;
Mem[50] = 32'h1862F800; Mem[51] = 32'h0ce77800;
Mem[52] = 32'h0ce77800; Mem[53] = 32'h0ce77800;
Mem[54] = 32'h1BE1F800; Mem[55] = 32'h0ce77800;
Mem[56] = 32'h0ce77800; Mem[57] = 32'h0ce77800;
Mem[58] = 32'h28190005; Mem[59] = 32'h281A0006;
Mem[60] = 32'h0ce77800; Mem[61] = 32'h0ce77800;
Mem[62] = 32'h0ce77800; Mem[63] = 32'h03A0C000;
Mem[64] = 32'h0ce77800; Mem[65] = 32'h0ce77800;
Mem[66] = 32'h0ce77800; Mem[67] = 32'h1F1AC000;
Mem[68] = 32'h0ce77800; Mem[69] = 32'h0ce77800;
Mem[70] = 32'h0ce77800; Mem[71] = 32'h1FD9F000;
```

```
Mem[72] = 32'h0ce77800; Mem[73] = 32'h0ce77800;
Mem[74] = 32'h0ce77800; Mem[75] = 32'h1C8C2000;
Mem[76] = 32'h0ce77800; Mem[77] = 32'h0ce77800;
Mem[78] = 32'h0ce77800; Mem[79] = 32'h1FEDF800;
Mem[80] = 32'h0ce77800; Mem[81] = 32'h0ce77800;
Mem[82] = 32'h0ce77800; Mem[83] = 32'h1C6B1800;
Mem[84] = 32'h0ce77800; Mem[85] = 32'h0ce77800;
Mem[86] = 32'h0ce77800; Mem[87] = 32'h1C4A1000;
Mem[88] = 32'h0ce77800; Mem[89] = 32'h0ce77800;
Mem[90] = 32'h0ce77800; Mem[91] = 32'h0F1EB800;
Mem[92] = 32'h0ce77800; Mem[93] = 32'h0ce77800;
Mem[94] = 32'h0ce77800; Mem[95] = 32'h0EE4B800;
Mem[96] = 32'h0ce77800; Mem[97] = 32'h0ce77800;
Mem[98] = 32'h0ce77800; Mem[99] = 32'h0EFFF800;
Mem[100]= 32'h0ce77800; Mem[101]= 32'h0ce77800;
Mem[102]= 32'h0ce77800; Mem[103]= 32'h0C7FF800;
Mem[104]= 32'h0ce77800; Mem[105]= 32'h0ce77800;
Mem[106]= 32'h0ce77800; Mem[107]= 32'h0C5FF800;
Mem[108]= 32'h0ce77800; Mem[109]= 32'h0ce77800;
Mem[110]= 32'h0ce77800; Mem[111]= 32'h0C3FF800;
Mem[112]= 32'h0ce77800; Mem[113]= 32'h0ce77800;
Mem[114]= 32'h0ce77800; Mem[115] = 32'hfc000000;
```

The major operations involved are:

- Loading input data
- Extracting data bits
- Calculating parity bits
- Forming the final Hamming code
- Storing the encoded output

9.1 Loading Input and Constants

The program first loads:

- Input data
- Mask values
- Shift values

The input binary data is stored in a register.

Example instructions:

```
ADDI R1, R0, data
ADDI R20, R0, 1
```

Where:

- R1 stores input data
- R20 stores mask value 1

Additional registers store shift values required for bit extraction.

9.2 Data Bit Extraction

The individual data bits are extracted using:

- Shift Right Logical (SRL)
- AND operations

The AND operation isolates the least significant bit after shifting.

Extraction of D1

```
AND R2, R1, R20
```

Extraction of D2

```
SRL R9, R1, R21
AND R3, R9, R20
```

Extraction of D3

```
SRL R9, R1, R22
AND R4, R9, R20
```

Extraction of D4

```
SRL R9, R1, R23
AND R5, R9, R20
```

After extraction:

- R2–R5 contain data bits D1–D4

9.3 Parity Bit Calculation

The parity bits are calculated using XOR operations.

Calculation of P1

```
XOR R10, R2, R3
XOR R6, R10, R5
```

Equation:

$$P1 = D1 \oplus D2 \oplus D4$$

Calculation of P2
XOR R11, R2, R4
XOR R7, R11, R5

Equation:

$$P2 = D1 \oplus D3 \oplus D4$$

Calculation of P4

XOR R12, R3, R4
XOR R8, R12, R5

Equation:

$$P4 = D2 \oplus D3 \oplus D4$$

The XOR operation generates parity bits required for Hamming encoding.

9.4 Formation of Final Hamming Code

After parity generation, all bits are arranged in the required Hamming structure.

Position	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Bit	P1	P2	D1	P4	D2	D3	D4

The processor uses:

- Shift Left Logical (SLL)
- OR operations

to place each bit in the correct position.

Example:

SLL R13, R6, shift
OR R14, R13, R7

Finally:

- The generated Hamming code is stored in output register R31.

9.5 Register Usage

Register	**Purpose**
R1	Input Data
R2–R5	Extracted Data Bits
R6	Parity Bit P1
R7	Parity Bit P2
R8	Parity Bit P4
R13–R15	Intermediate Shift Operations
R31	Final Hamming Code

9.6 Program Termination

After successful Hamming code generation, the processor executes the halt instruction.

HLT

The halt signal indicates completion of execution.

10. ASSEMBLY LANGUAGE PROGRAM FOR ERROR DETECTION

The error detection program checks whether the received Hamming code contains any error. The program calculates syndrome bits and determines the exact error location.

```
// --- 71 OPERATIONS WITH BOTH NOPs AND LOGIC FIXES ---
    Mem[0] = 32'h281b0001;    // R27 = 1 (Mask)
    Mem[1] = 32'h28150001;    // R21 = 1
    Mem[2] = 32'h28160002;    // R22 = 2
    Mem[3] = 32'h28170003;    // R23 = 3
    Mem[4] = 32'h28180004;    // R24 = 4
    Mem[5] = 32'h28190005;    // R25 = 5
    Mem[6] = 32'h281a0006;    // R26 = 6
    Mem[7]=32'h0ffff800;      Mem[8]=32'h0ffff800;      Mem[9]=32'h0ffff800;
Mem[10]=32'h0ffff800;

    // Bit 1 -> R2
    Mem[11] = 32'h20204800;   // SRL R9, R1, R0
    Mem[12]=32'h0ffff800; Mem[13]=32'h0ffff800; Mem[14]=32'h0ffff800;
    Mem[15] = 32'h093b1000;   // AND R2, R9, R27
    // Bit 2 -> R3
    Mem[16] = 32'h20354800;   // SRL R9, R1, R21
    Mem[17]=32'h0ffff800; Mem[18]=32'h0ffff800; Mem[19]=32'h0ffff800;
    Mem[20] = 32'h093b1800;   // AND R3, R9, R27
    // Bit 3 -> R4
    Mem[21] = 32'h20364800;   // SRL R9, R1, R22
    Mem[22]=32'h0ffff800; Mem[23]=32'h0ffff800; Mem[24]=32'h0ffff800;
    Mem[25] = 32'h093b2000;   // AND R4, R9, R27
    // Bit 4 -> R5
    Mem[26] = 32'h20374800;   // SRL R9, R1, R23
    Mem[27]=32'h0ffff800; Mem[28]=32'h0ffff800; Mem[29]=32'h0ffff800;
    Mem[30] = 32'h093b2800;   // AND R5, R9, R27
    // Bit 5 -> R6
    Mem[31] = 32'h20384800;   // SRL R9, R1, R24
    Mem[32]=32'h0ffff800; Mem[33]=32'h0ffff800; Mem[34]=32'h0ffff800;
    Mem[35] = 32'h093b3000;   // AND R6, R9, R27
```

```
// Bit 6 -> R7
Mem[36] = 32'h20394800;     // SRL R9, R1, R25
Mem[37]=32'h0ffff800; Mem[38]=32'h0ffff800; Mem[39]=32'h0ffff800;
Mem[40] = 32'h093b3800;     // AND R7, R9, R27
// Bit 7 -> R8
Mem[41] = 32'h203a4800;     // SRL R9, R1, R26
Mem[42]=32'h0ffff800; Mem[43]=32'h0ffff800; Mem[44]=32'h0ffff800;
Mem[45] = 32'h093b4000;     // AND R8, R9, R27

// Syndrome
Mem[46] = 32'h18435000; Mem[47] = 32'h18855800;
Mem[48] = 32'h18436000; Mem[49] = 32'h18c76800;
Mem[50] = 32'h18447000; Mem[51] = 32'h18c87800;
Mem[52]=32'h0ffff800; Mem[53]=32'h0ffff800; Mem[54]=32'h0ffff800;
Mem[55] = 32'h194b9000; Mem[56] = 32'h198d8800; Mem[57] = 32'h19cf8000;

// Concatenation
Mem[58]=32'h0ffff800; Mem[59]=32'h0ffff800; Mem[60]=32'h0ffff800;
Mem[61] = 32'h1e569800; Mem[62] = 32'h1e35a000;
Mem[63]=32'h0ffff800; Mem[64]=32'h0ffff800; Mem[65]=32'h0ffff800;
Mem[66] = 32'h0e74e000;
Mem[67]=32'h0ffff800; Mem[68]=32'h0ffff800; Mem[69]=32'h0ffff800;
Mem[70] = 32'h0f90e800;
Mem[71] = 32'hfc000000; // HLT
```

The major operations involved are:

- Loading received Hamming code
- Extracting all seven bits
- Calculating syndrome bits
- Combining syndrome values
- Identifying error position

10.1 Loading Input and Constants

The received Hamming code is loaded into the processor register.

Example:

ADDI R1, R0, 76

Where:

- 76 represents Hamming code 1001100

Additional registers store:

- Mask values
- Shift values

10.2 Bit Extraction

All seven bits are extracted individually using:

- SRL operation
- AND masking

Extract Bit 7

```
SRL R9, R1, R0
AND R2, R9, R27
```

Extract Bit 6

```
SRL R9, R1, R21
AND R3, R9, R27
```

Similarly:

- All remaining bits are extracted into separate registers.

10.3 Syndrome Calculation

Three syndrome bits are calculated.

Calculation of S3

```
XOR R10, R2, R3
XOR R11, R4, R5
XOR R18, R10, R11
```

Equation:

$$S3 = b1 \oplus b2 \oplus b3 \oplus b4$$

Calculation of S2

```
XOR R12, R2, R3
XOR R13, R6, R7
XOR R17, R12, R13
```

Equation:

$$S2 = b1 \oplus b2 \oplus b5 \oplus b6$$

Calculation of S1

XOR R14, R2, R4
XOR R15, R6, R8
XOR R16, R14, R15

Equation:

$$S1 = b1 \oplus b3 \oplus b5 \oplus b7$$

10.4 Syndrome Concatenation

The syndrome bits are combined to form the final syndrome.

SLL R19, R18, R22
SLL R20, R17, R21
OR R28, R19, R20
OR R29, R28, R16

Final syndrome:

$$S = (S3\ S2\ S1)$$

The syndrome is stored in register R29.

10.5 Syndrome Interpretation

Syndrome	Meaning
000	No Error
001	Error in Bit 1
010	Error in Bit 2
011	Error in Bit 3
100	Error in Bit 4
101	Error in Bit 5
110	Error in Bit 6
111	Error in Bit 7

10.6 Register Usage

Register	Purpose
R1	Input Hamming Code
R2–R8	Extracted Bits
R16	Syndrome Bit S1
R17	Syndrome Bit S2
R18	Syndrome Bit S3
R29	Final Syndrome

10.7 Program Termination

After syndrome calculation:

- The processor executes halt instruction.

HLT

Execution completion is indicated through the halt signal.

10.8 Error Injection Algorithm

After generating the valid 7-bit Hamming code, an error is intentionally introduced to verify the error detection capability of the system. A predefined error mask is applied to the generated Hamming code using the XOR operation.

Steps involved:

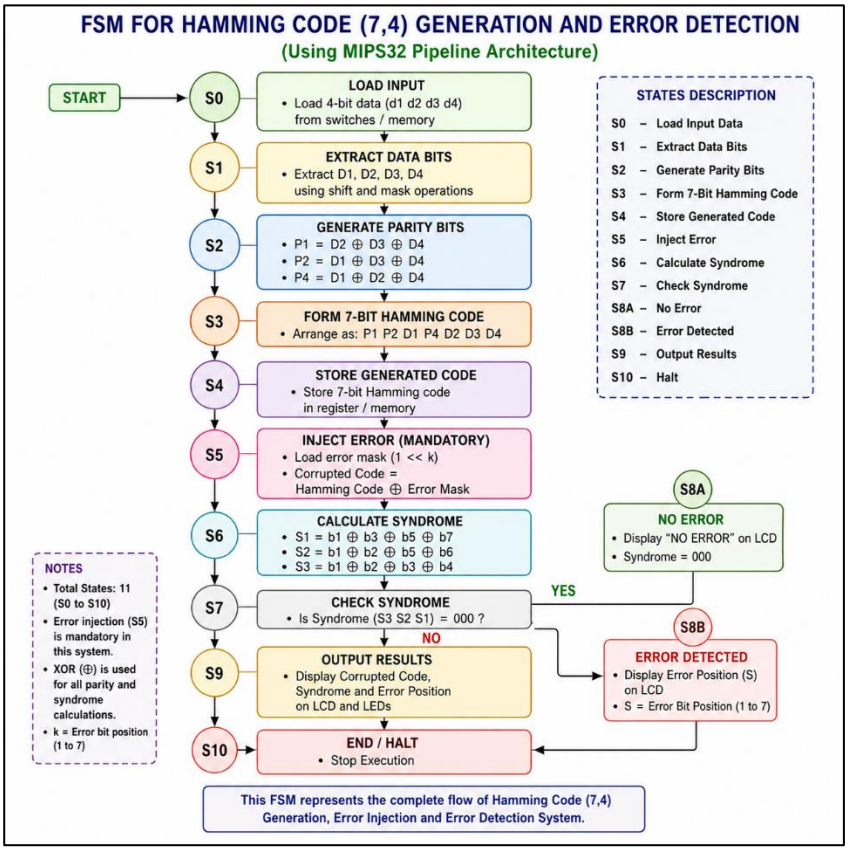
1. Generate the valid 7-bit Hamming code from the 4-bit input data.
2. Load the error mask corresponding to the bit position where an error is to be introduced.
3. Perform XOR operation between the generated Hamming code and the error mask.
4. The resulting code becomes the corrupted Hamming code.
5. The corrupted code is then provided as input to the syndrome detection module.
6. Syndrome calculation identifies the exact bit position where the error was introduced.

Mathematically,

Corrupted Hamming Code = Generated Hamming Code XOR Error Mask

This method allows controlled single-bit error insertion and helps verify the correctness of the Hamming code error detection process.

Decimal	Data (d1 d2 d3 d4)	Transmitted (P1 P2 D1 P4 D2 D3 D4)
0	0000	0000000
1	0001	1101001
2	0010	0101010
3	0011	1000011
4	0100	1001100
5	0101	0100101
6	0110	1100110
7	0111	0001111
8	1000	1110000
9	1001	0011001
10	1010	1011010
11	1011	0110011
12	1100	0111100
13	1101	1010101
14	1110	0010110
15	1111	1111111



11. HARDWARE IMPLEMENTATION

The complete system is implemented on FPGA hardware using Verilog HDL. The hardware setup includes:

- DIP switches for input
- LEDs for syndrome display
- LCD display for error information

Input Section

The 4-bit binary input is provided using:

- FPGA DIP switches

The switch values are directly connected to the processor input.

Processing Section

The MIPS32 processor performs:

- Hamming code generation
- Syndrome calculation
- Error detection

The processor executes ALP instructions stored in instruction memory.

Output Section

The outputs are displayed through:

- LEDs
- LCD display

LED Output

Displays:

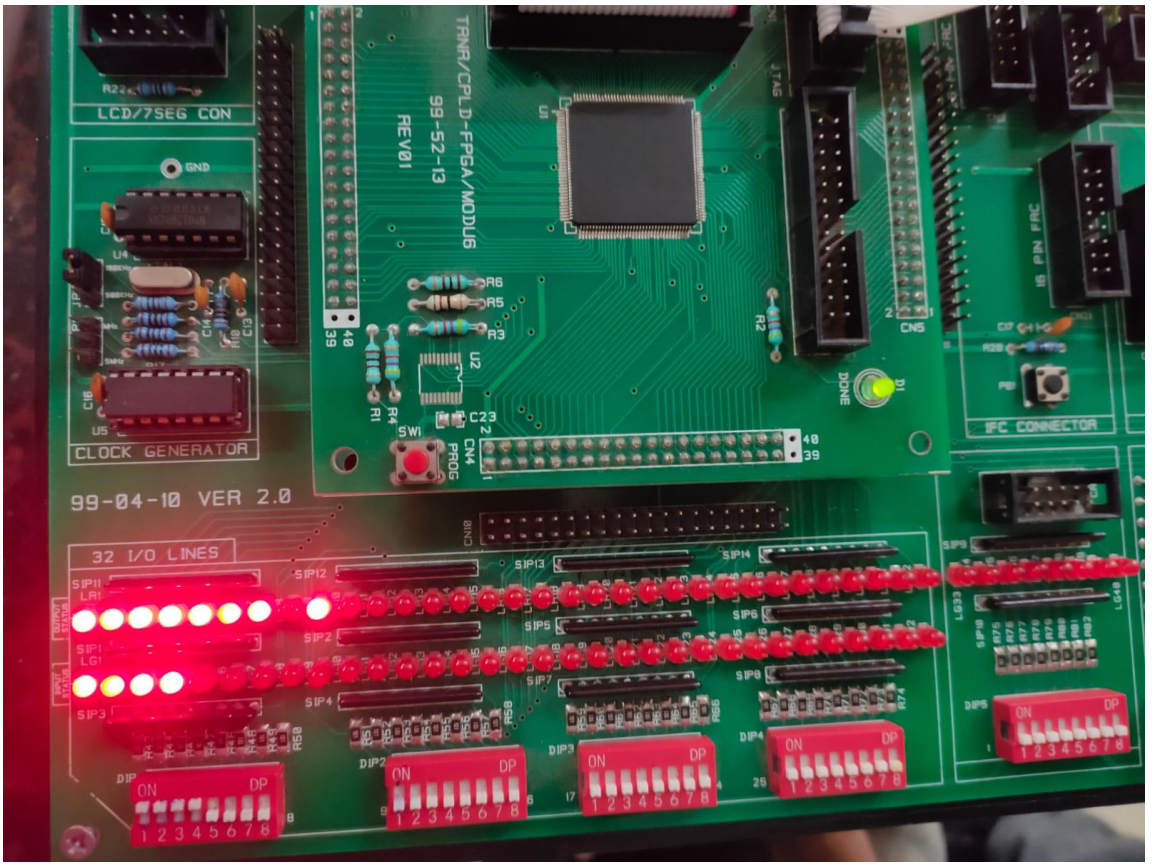
- Syndrome bits
- Hamming output values

LCD Output

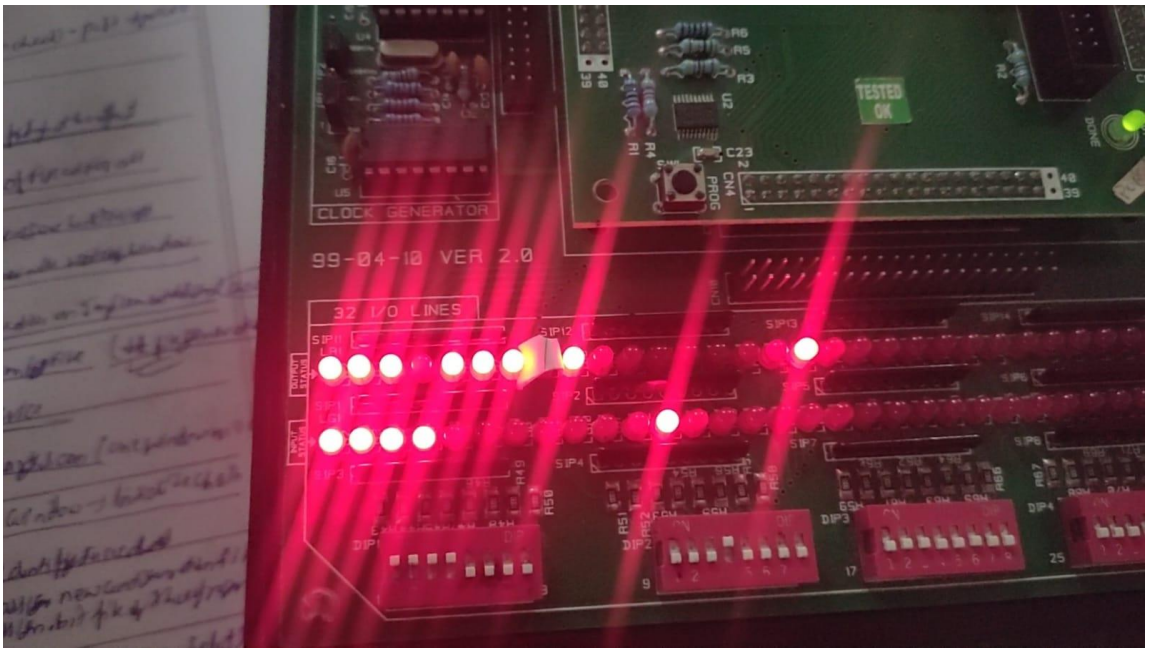
Displays:

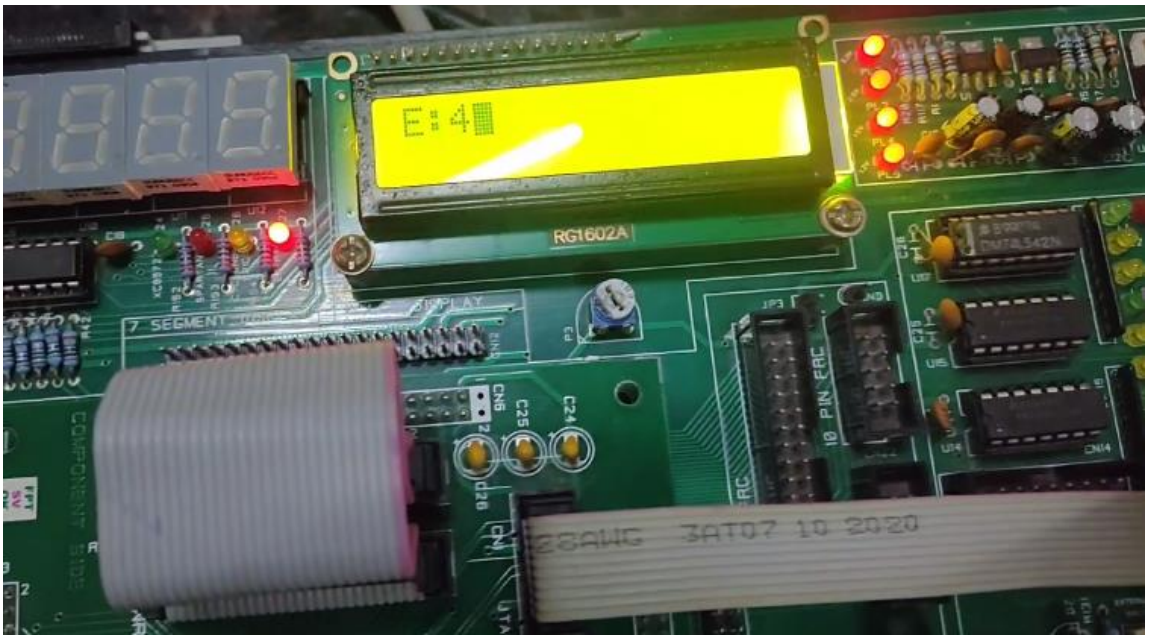
- "No Error"
  or
- "Err:X"

depending on syndrome result.



No Error on Codeword 1111111





Error at Bit Position 4 on Codeword 1111111

FPGA Implementation

The Verilog design is synthesized and implemented on FPGA hardware.

The FPGA implementation verifies:

- Real-time execution
- Correct parity generation
- Correct error detection

12. RESULTS AND WAVEFORMS

The project is verified through:

- Software simulation
- Hardware implementation

Waveforms confirm:

- Correct parity generation
- Correct syndrome calculation
- Proper pipeline execution

12.1 Hamming Code Generation Result

Example 1:

Input Data:

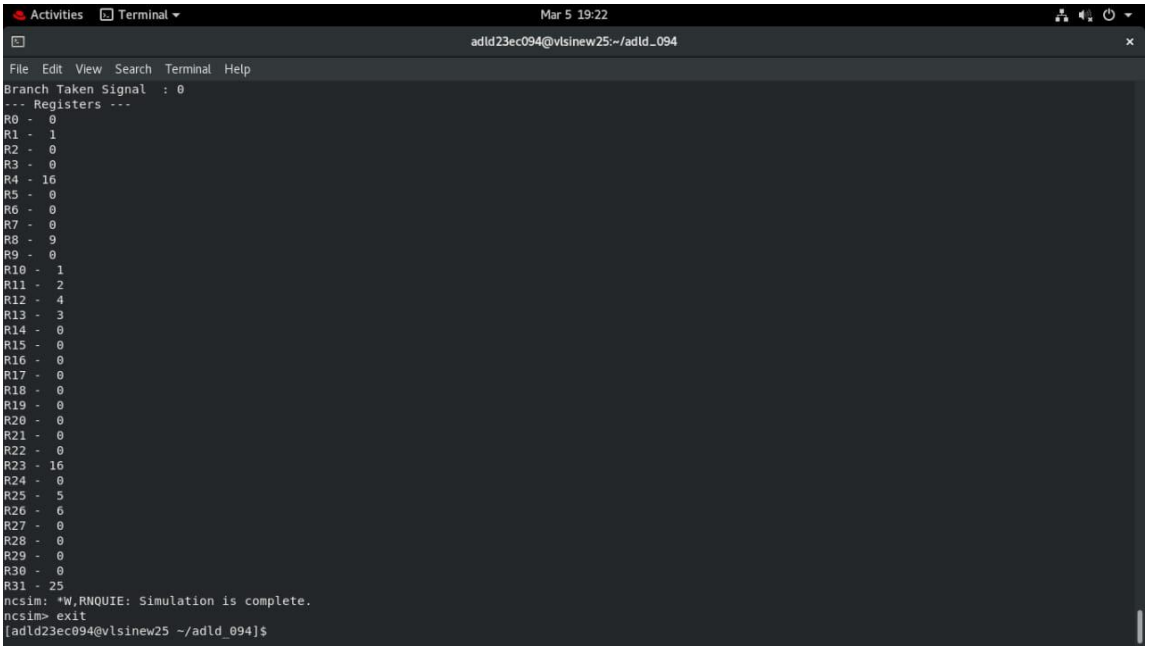
1001

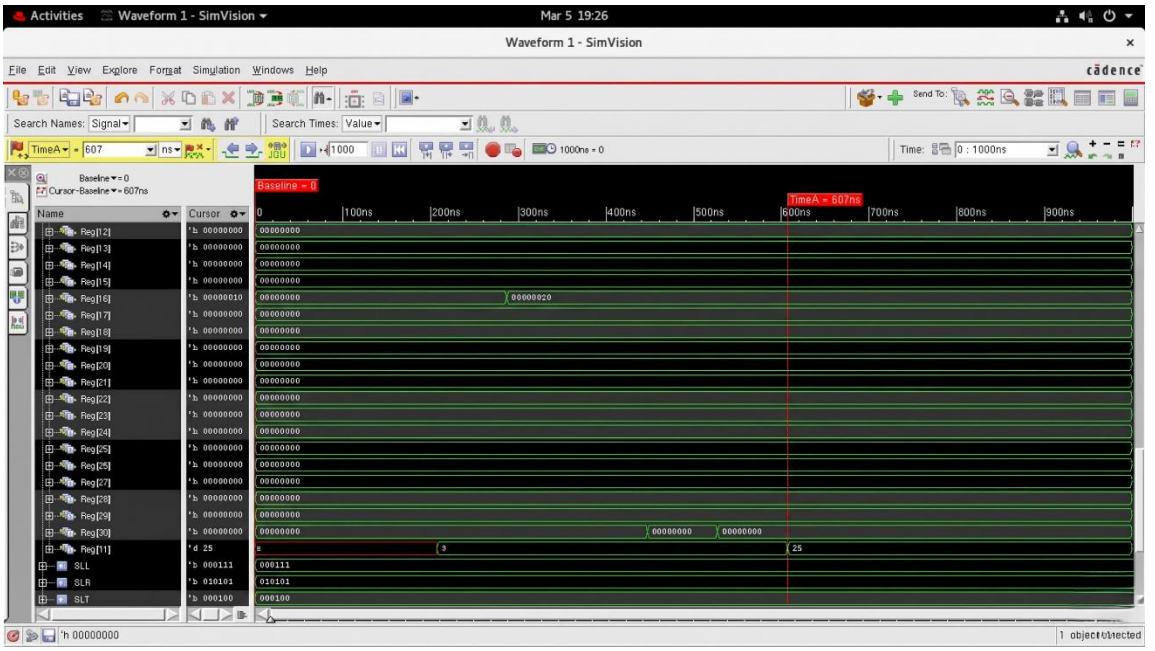
Generated Hamming Code:

0011001

Decimal Equivalent:

25





Example 2:

Input Data:

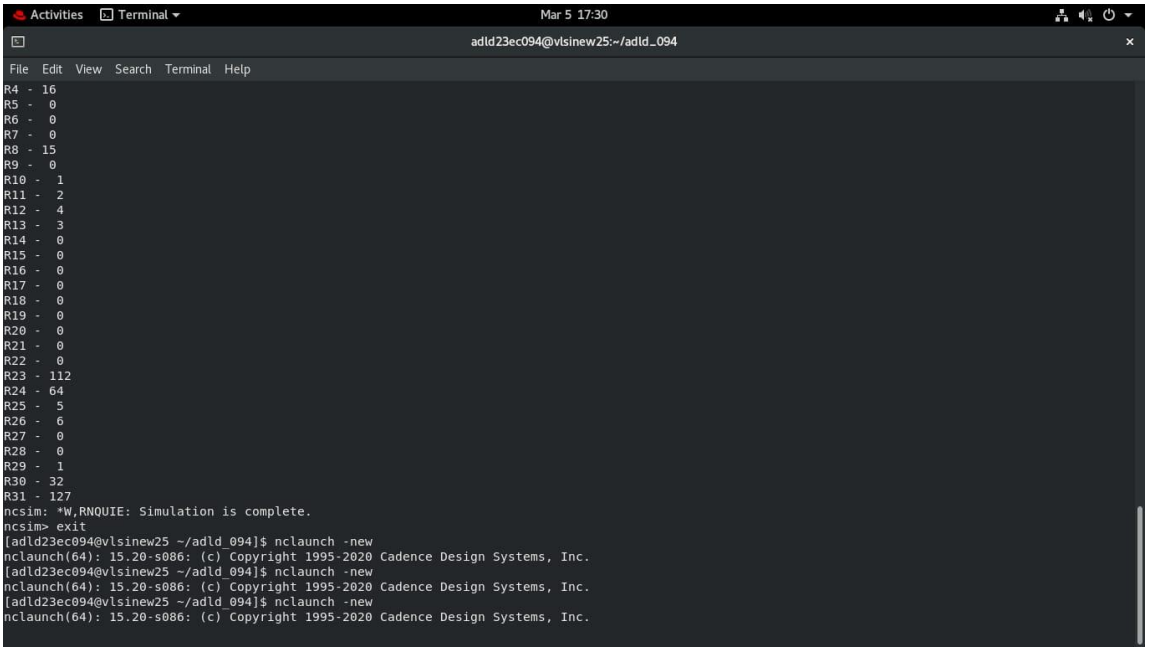
1111

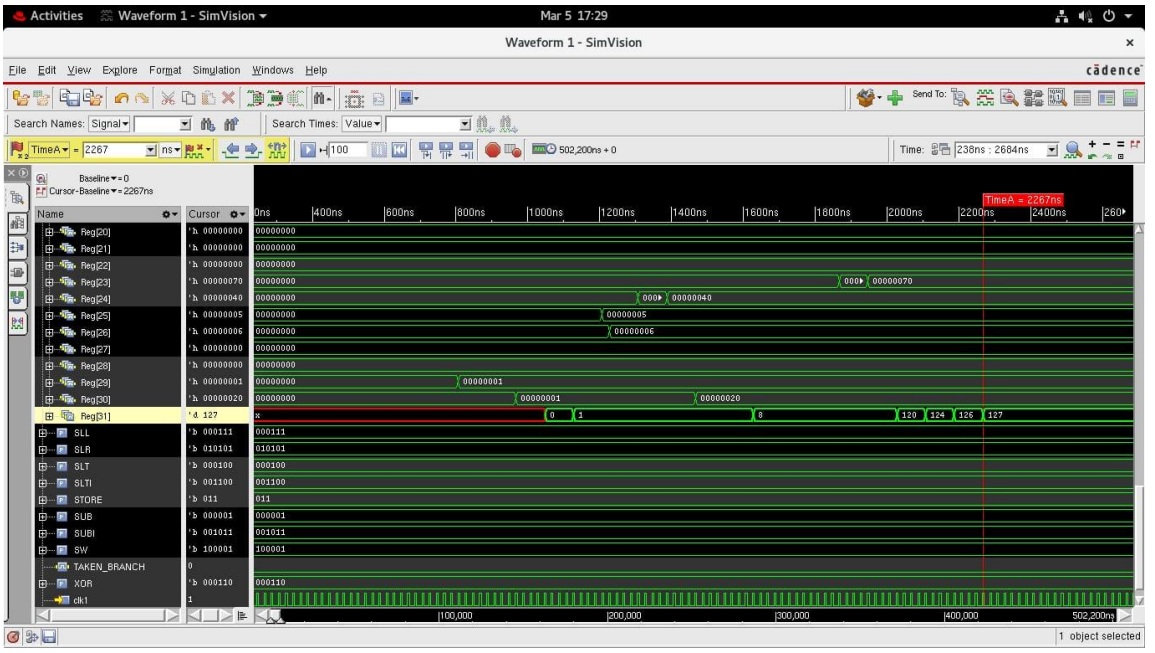
Generated Hamming Code:

1111111

Decimal Equivalent:

127





Waveforms confirm correct execution of:

- XOR operations
- Shift operations
- Final output storage

12.2 Error Detection Result

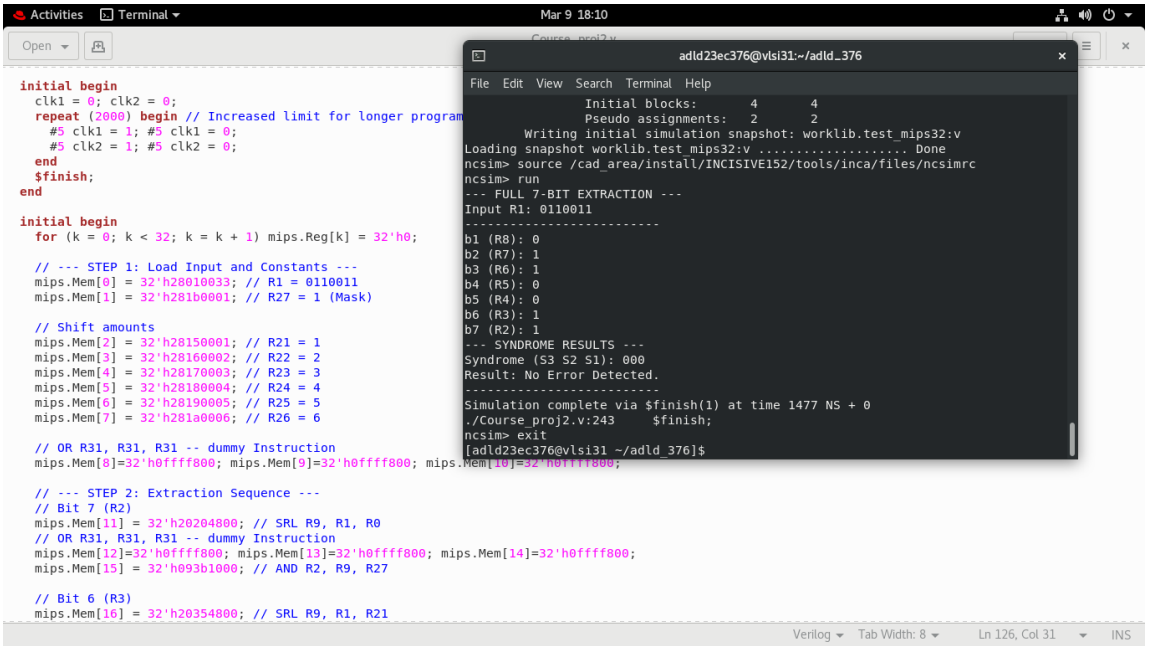
Example 1: No Error

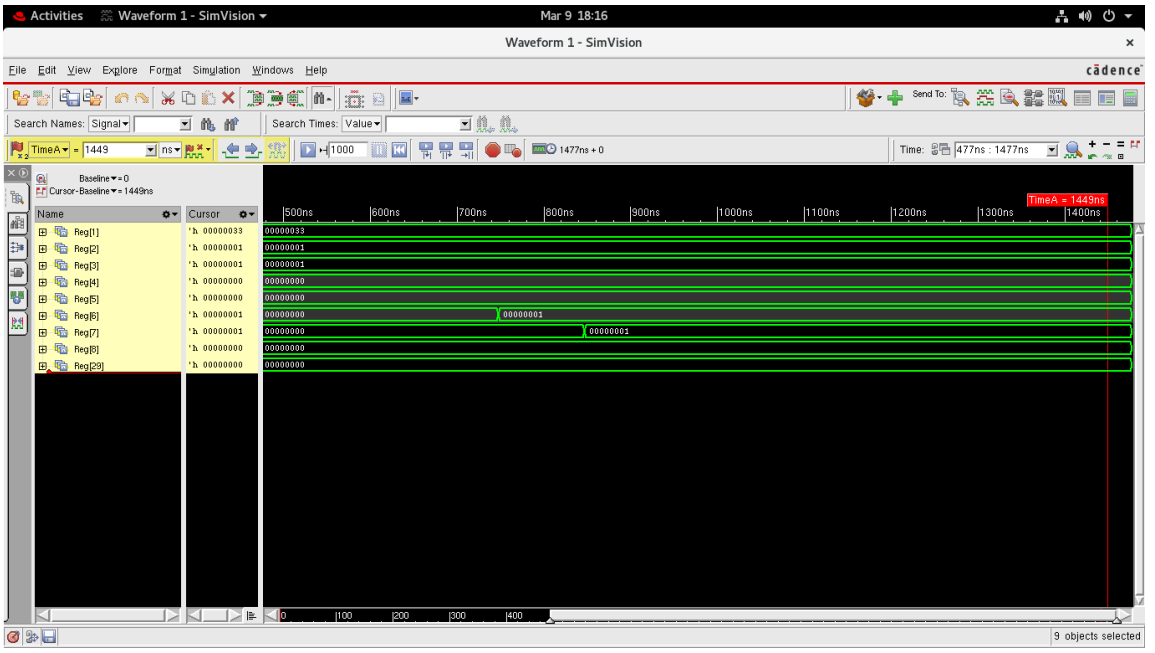
Original Hamming Code:

0110011

Calculated Syndrome:

000





Example 2: Error

Original Hamming Code:

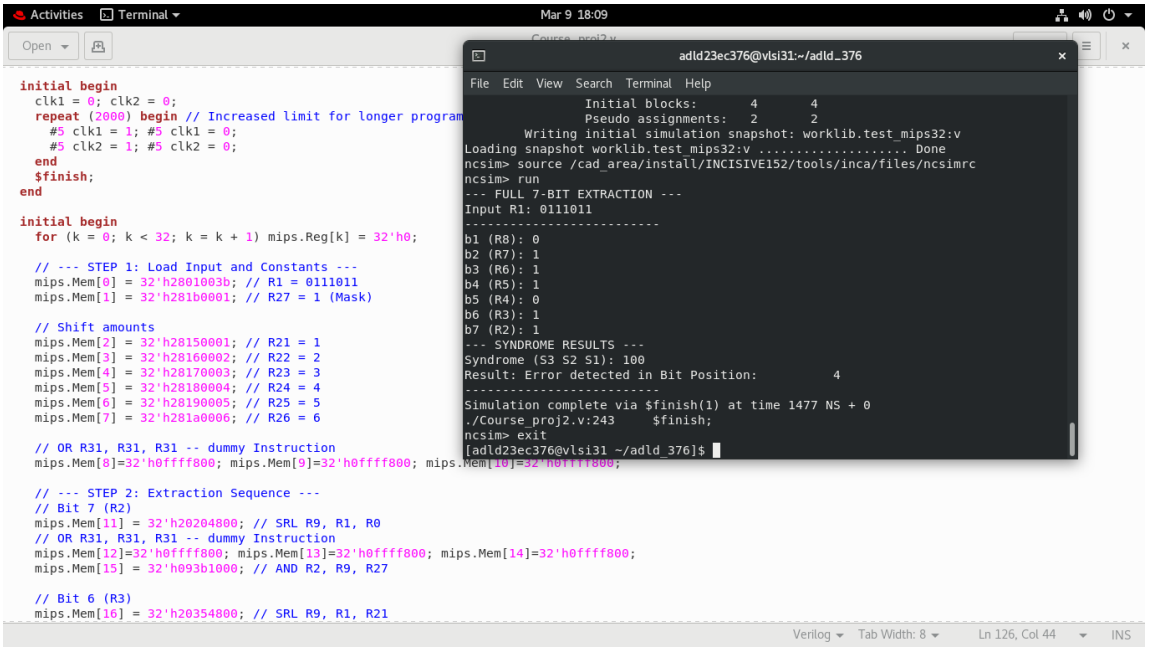
0110011

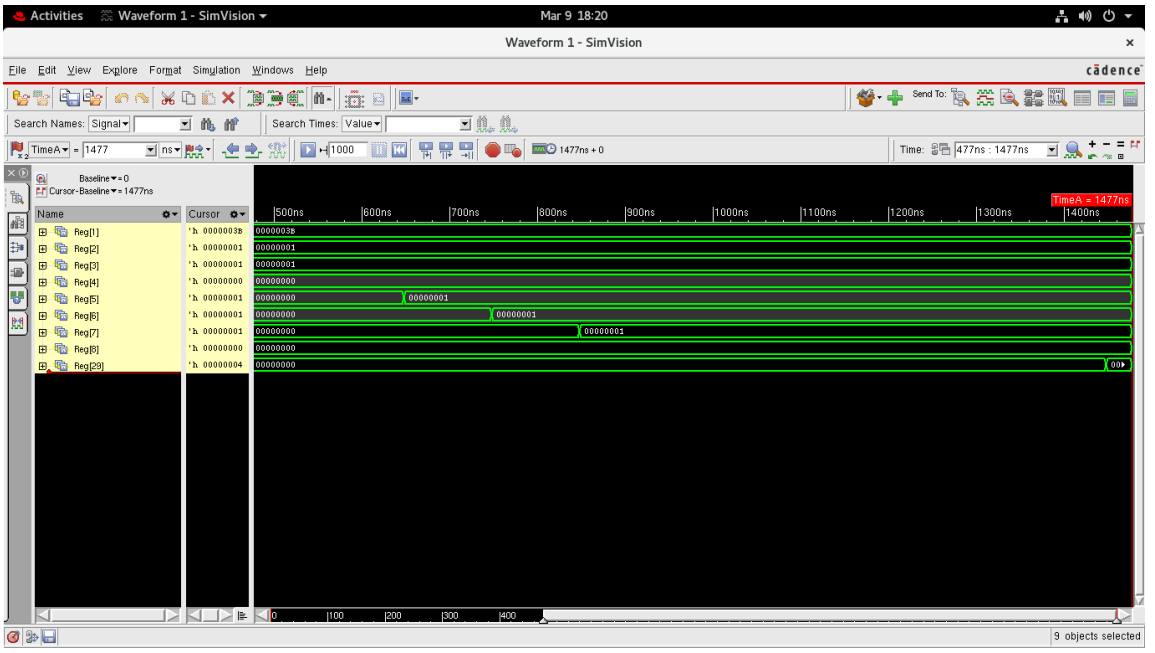
Error Introduced:

0111011

Calculated Syndrome:

100





Therefore:

- Error exists at bit position 4

12.3 Hardware Result

Hardware implementation successfully demonstrates:

- Input through switches
- Output through LEDs
- LCD display messages

The FPGA system performs:

- Real-time Hamming conversion
- Real-time syndrome detection

13. ADVANTAGES OF THE PROJECT

• Detects Single-Bit Errors

The system accurately detects single-bit transmission errors.

• Fast Execution Using Pipeline

Pipeline architecture improves processor performance.

• Efficient Hardware Utilization

All pipeline stages operate simultaneously.

• Real-Time Hardware Demonstration

FPGA implementation demonstrates practical operation.

• Reliable Communication Support

Improves data reliability in communication systems.

• Modular Verilog Design

Modules can be reused for advanced systems.

14. APPLICATIONS

Hamming Code and pipelined processors are widely used in digital systems.

Applications include:

• Digital Communication Systems

Used for reliable data transmission.

• Computer Memory Systems

Used in RAM error correction systems.

• Embedded Systems

Used in fault-tolerant processors.

• Networking Systems

Used for packet error detection.

• FPGA and VLSI Design

Used for hardware reliability testing.

• Wireless Communication

Used in noisy communication channels.

15. CONCLUSION

This project successfully demonstrates the implementation of Hamming Code (7,4) generation and single-bit error detection using a pipelined MIPS32 processor implemented in Verilog HDL.

The system generates parity bits, forms the 7-bit Hamming code, calculates syndrome values, and identifies the exact error position accurately. The use of a 5-stage pipeline architecture improves execution efficiency through parallel instruction processing.

Both software simulation and FPGA hardware implementation confirm the correctness of the design. The LCD and LED outputs provide clear visualization of generated Hamming codes and detected errors.

The project effectively combines:

- Digital communication concepts
- Error detection techniques
- Assembly language programming
- Pipeline processor architecture
- Verilog HDL implementation

Thus, the project provides a practical understanding of reliable data transmission and processor-based hardware design.